

ML Project Deployment — Phase 1

Project Structure & Backend Setup — Final Report

Project: Predicting Employee Burnout and Resignation Risk Using Machine Learning on Workplace Behavioral Data

Milestone: 4 — ML Project Deployment, Phase 1

Name: Fajar Khalid

Date: August 3, 2026

1. Objective

This phase transforms the trained machine learning model developed in Milestones 1-3 (dataset preprocessing, feature engineering, and model training/evaluation, all originally in notebook form) into a clean, modular, deployment-ready project structure with a basic working interface. The goal was not to build the final polished application, but to prove the project is organized correctly and the backend functions end-to-end without retraining the model on every run.

2. Project Structure

```
employee-attrition-deployment/
├── data/
│   └── HR_Attrition_synthetic.csv      (demo dataset)
├── models/
│   └── attrition_model_bundle.joblib   (serialized model + preprocessing)
├── src/
│   ├── data_preprocessing.py          (cleaning, encoding)
│   ├── feature_engineering.py         (engineered features, Milestone 2)
│   ├── model_loader.py                (loads the Joblib bundle)
│   ├── predict_pipeline.py            (end-to-end inference)
│   └── utils.py                       (risk-tier logic, validation)
├── train.py                           (trains + serializes the model)
├── app.py                             (basic Streamlit demo interface)
├── requirements.txt
└── README.md
```

Each script has a single responsibility, and both training (`train.py`) and inference (`predict_pipeline.py`) import the same `data_preprocessing.py` and `feature_engineering.py` functions — so preprocessing logic exists in exactly one place and cannot silently drift between training and inference.

3. Technology Stack

- Language: Python 3
- Data handling: Pandas, NumPy
- Machine Learning: Scikit-learn (Logistic Regression, StandardScaler, LabelEncoder), Imbalanced-learn (SMOTE-Tomek)
- Model persistence: Joblib
- Application framework: Streamlit (basic demo interface for this phase)
- Version control: Git & GitHub

4. Model Training & Serialization

train.py reproduces the Milestone 1-3 pipeline in modular form: preprocessing (drop constant columns, label-encode binary fields, one-hot encode multi-class fields) → feature engineering (TenureRatio, PromotionGap, AvgSatisfactionScore, ManagerStabilityRatio, IsFrequentJobHopper, log-transformed income) → an 80/20 stratified train/test split → StandardScaler → SMOTE-Tomek balancing on the training set only → Logistic Regression, the best-performing model identified in Milestone 3.

The trained model, fitted scaler, fitted label encoders, and the exact training-time feature column order are bundled together and saved with Joblib as a single object. Bundling all of these together — rather than the model alone — was a deliberate design choice: it guarantees that inference-time preprocessing can never fall out of sync with what the model was actually trained on.

Evaluation results (on held-out test data)

Metric	Value
Accuracy	0.7381
Precision	0.3679
Recall	0.7959
F1-Score	0.5032

These results are consistent with the Logistic Regression performance reported in Milestone 3, confirming the refactored, modular pipeline reproduces the original notebook results.

5. Basic Application

app.py is a minimal Streamlit interface that loads the serialized model bundle once (cached with `@st.cache_resource` so it is never retrained while the app is running), collects a single employee's data through a form, and returns a resignation-risk probability, risk tier (Low/Medium/High), and prediction using the same `predict_pipeline.py` used for batch scoring. The app was started locally and confirmed to load successfully (HTTP 200, no errors) before submission.

6. Issues Faced & How They Were Resolved

6.1 Inference-time column mismatch

One-hot encoding a single employee record at inference time produces different (and fewer) dummy columns than the full training set — for example, if a category present during training doesn't appear in that one row. This was resolved by saving the exact training-time `feature_columns` list inside the model bundle and calling `DataFrame.reindex(columns=feature_columns, fill_value=0)` before scaling, guaranteeing the inference feature vector always has the identical shape and column order the model was trained on.

6.2 Avoiding retraining on every app run

Streamlit reruns the entire script on every user interaction. Loading the model bundle directly inside `app.py` without caching would have reloaded it from disk (and risked accidental retraining logic) on every form submission. This was solved with Streamlit's `@st.cache_resource` decorator around the model-loading function, so the bundle is loaded from disk exactly once per session.

6.3 Division-by-zero in feature engineering

The TenureRatio feature ($\text{YearsAtCompany} / \text{TotalWorkingYears}$) breaks for employees with 0 total working years — a realistic case for new graduates. This was fixed by replacing 0 with a small constant (0.5) only in the denominator, preserving the ratio's meaning without crashing the pipeline.

6.4 Keeping train/inference preprocessing identical

Early versions of this project had slightly different encoding logic written separately for training versus single-row inference, which is a common source of silent bugs in deployed ML systems. This was resolved by refactoring all preprocessing into shared functions in `data_preprocessing.py` that both `train.py` and `predict_pipeline.py` import and call directly — there is now exactly one implementation of the preprocessing logic.

7. Verification

Before submission, the full pipeline was tested end-to-end:

- `train.py` ran successfully end-to-end, producing the metrics shown in Section 4.
- `predict_pipeline.py` was tested directly against a clearly high-risk profile (young, low income, frequent overtime, low satisfaction) and a clearly low-risk profile (senior, high income, high satisfaction, long tenure) — the model correctly scored the high-risk profile at 99.3% and the low-risk profile at 0.7% resignation probability.
- Batch prediction (`predict_batch`) was tested on a small dataframe of both profiles and returned correctly labeled `ResignationProbability` and `RiskTier` columns.
- `app.py` was started with ``streamlit run app.py`` and confirmed to load and serve the interface successfully without errors.

8. About the Data

The development environment does not have direct access to Kaggle, so `train.py` generates a synthetic dataset that mirrors the exact column schema and documented behavioral patterns of the IBM HR Analytics Employee Attrition dataset used in Milestones 1-3 (e.g., overtime and low satisfaction increase resignation probability; tenure, income, and stock options reduce it). `train.py` fully supports swapping in the real dataset via a `--real-data` command-line flag, with no other code changes required.

9. Next Steps

- Build out the full web application interface beyond this basic demo form
- Add batch CSV upload directly to the Streamlit/web interface
- Containerize the application with Docker for deployment
- Deploy to a hosting platform (e.g., Streamlit Community Cloud, Render, or similar)